Name: Gaurang Vaghela
Rollno: BEAD-22561
CI Practical 6
Problem Statement: Implementation of Clonal selection algorithm using Python.

Code:

```
import numpy as np
import random

# Fitness function (minimize x^2)
def fitness(x):
    return x**2

# Initialize population
def initialize_population(size):
    return [random.uniform(-10, 10) for _ in range(size)]

# Calculate affinity (inverse of fitness)
def calculate_affinity(pop):
    return [(x, 1 / (1 + fitness(x))) for x in pop]

# Select best antibodies
def select_antibodies(pop, affinity, num_selected):
    sorted_pop = sorted(zip(pop, affinity), key=lambda x: x[1][1], reverse=True)
    return [x[0] for x in sorted_pop[:num_selected]]

# Clone antibodies
def clone_antibodies(selected):
    clones = []
    for i, x in enumerate(selected):
        num_clones = len(selected) - i  # higher rank → more clones
        clones.extend([x] * num_clones)
    return clones

# Mutation
def mutate(clones, mutation_rate):
    mutated = []
    for x in clones:
        if random.random() < mutation_rate:
            x = x + random.uniform(-1, 1)
        mutated.append(x)
    return mutated
```

```
# Select next generation
def select_next_generation(pop, mutated, size):
    combined = pop + mutated
    combined.sort(key=lambda x: fitness(x))  # minimize
    return combined[:size]

# Main CSA Algorithm
def clonal_selection(pop_size=10, mutation_rate=0.2, iterations=20):
    population = initialize_population(pop_size)
    for i in range(iterations):
        affinity = calculate_affinity(population)
        selected = select_antibodies(population, affinity, pop_size // 2)
        clones = clone_antibodies(selected)
        mutated = mutate(clones, mutation_rate)
        population = select_next_generation(population, mutated, pop_size)
        best = min(population, key=fitness)
        print(f"Iteration {i}: Best = {best:.4f}, Fitness = {fitness(best):.4f}")
    return min(population, key=fitness)

# Run algorithm
best_solution = clonal_selection()
print("\nBest Solution Found:", best_solution)
print("Minimum Value:", fitness(best_solution))
```

OUTPUT:

```
vboxuser@Ubuntu-24: ~/Desktop/CI6
vboxuser@Ubuntu-24:~/Desktop/CI6$ python3 CI6.py
Iteration 0: Best = -0.9691, Fitness = 0.9391
Iteration 1: Best = -0.0837, Fitness = 0.0070
Iteration 2: Best = -0.0837, Fitness = 0.0070
Iteration 3: Best = -0.0837, Fitness = 0.0070
Iteration 4: Best = -0.0837, Fitness = 0.0070
Iteration 5: Best = -0.0837, Fitness = 0.0070
Iteration 6: Best = -0.0837, Fitness = 0.0070
Iteration 7: Best = -0.0837, Fitness = 0.0070
Iteration 8: Best = -0.0765, Fitness = 0.0059
Iteration 9: Best = -0.0765, Fitness = 0.0059
Iteration 10: Best = -0.0765, Fitness = 0.0059
Iteration 11: Best = -0.0765, Fitness = 0.0059
Iteration 12: Best = -0.0049, Fitness = 0.0000
Iteration 13: Best = -0.0049, Fitness = 0.0000
Iteration 14: Best = -0.0049, Fitness = 0.0000
Iteration 15: Best = -0.0049, Fitness = 0.0000
Iteration 16: Best = -0.0049, Fitness = 0.0000
Iteration 17: Best = -0.0049, Fitness = 0.0000
Iteration 18: Best = -0.0049, Fitness = 0.0000
Iteration 19: Best = -0.0049, Fitness = 0.0000

Best Solution Found: -0.004873564592593027
Minimum Value: 2.3751631838176438e-05
vboxuser@Ubuntu-24:~/Desktop/CI6$
```